A

SRS AND DESIGN REPORT

ON

FAKE PROFILE DETECTOR

Submitted in partial fulfilment of requirements for the award of the Degree of

BACHELOR OF ENGINEERING

IN

CSE (Artificial Intelligence & Machine Learning)

Submitted by

D. Aditya	**245323748088**
G. Yashwanth	**245323748092**
B. Himesh Reddy	**245323748073**

Under the

guidance of

Mrs. Neena Sudheera,
Assistant Professor

NEIL GOGTE INSTITUTE OF TECHNOLOGY

Kachavanisingaram Village, Hyderabad, Telangana 500058.
Affiliated to Osmania University

Kachavanisingaram Village, Hyderabad, Telangana 500058

2025 – 2026

NEIL GOGTE INSTITUTE OF TECHNOLOGY
A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad–500088, Telangana.

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGCENCE & MACHINE LEARNING)

CERTIFICATE

This is to certify that this is a Bonafide record of the project report titled **FAKE PROFILE DETECTOR** which is being presented as project by D. Aditya, 245323748088, G. Yashwanth 245323748092, B. Himesh Reddy, 245323748073 in partial fulfilment for the award of the Degree of Bachelor of Engineering in Computer Science affiliated to the Osmania University, Hyderabad.

Internal Guide

Mrs. Neena Sudheera,
Assistant Professor

Head of Department

Dr K Vinuthna Reddy
Associate Professor

External Examiner

NEIL GOGTE INSTITUTE OF TECHNOLOGY
A unit of Keshav Memorial Technical Educational Society (KMTES)
(Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad)
D.No:10TC-111, Kachivanisingaram (V), Uppal, Hyderabad–500088, Telangana.

DECLARATION

We hereby declare that the results embodied in the dissertation **FAKE PROFILE DETECTOR** has been carried myself during the academic year 2025-26 as a partial fulfilment of the award of the Bachelor of Engineering Degree in Computer Science from Osmania University. We have not submitted this report to any other university or organization for the award of any other degree.

D. Aditya	245323748088
G. Yashwanth	245323748092
B. Himesh Reddy	245323748073

Comprehensive Project Execution Report: InstaGuard

Executive Summary

Social media platforms are increasingly plagued by automated bots and malicious fake profiles. These accounts are often used for identity theft, spamming, spreading disinformation, and inflating social engagement metrics. **InstaGuard** addresses this issue by leveraging supervised machine learning to classify accounts into two categories: **Real** (Class 0) or **Fake** (Class 1). By analyzing public metadata like follower counts, posting activity, name formatting, and bio content, InstaGuard provides a local web-based inference engine that predicts profile authenticity with **90.52% accuracy**. This report compiles the comprehensive lifecycle of the project, including scope definition, system design, implementation hurdles, performance metrics, and validation test cases.

Phase 1: Define Scope & Requirements

1. Project Objectives:

- **High Accuracy Classification:** Target a prediction accuracy of **>90%** on unseen test data using supervised machine learning.
- **Sub-Second Web Inference:** Provide a responsive Flask web interface that executes predictions and returns verdicts in less than 100 milliseconds.
- **Privacy-First Design:** Build the detector entirely around publicly observable profile metrics (follower counts, post counts, bio characteristics) rather than scraping private user data or media content.
- **Developer-Friendly Extensibility:** Organize the codebase into modular pipelines so new features, data sources, and models can be easily integrated.

2. Core Features:

- **Offline Training & Evaluation Pipeline:** A set of structured Python modules to ingest raw CSV data, perform feature engineering, split datasets into training and validation sets, train a classifier, and output detailed validation metrics.
- **Dynamic Feature Engineering:** Automate the extraction of advanced behavioral signals, such as follower-to-following ratios and posts-to-following ratios, which highlight anomalous user behavior.
- **Web Interface (InstaGuard Dashboard):** An interactive, responsive, and visually appealing web interface built using HTML5, CSS3, and JavaScript, featuring glassmorphism elements, loading animations, and dynamic result styling.
- **Serialized Model Inference Layer:** A lightweight server-side inference module that loads a pre-trained model file (*model.pkl*) into memory when Flask starts up to process and classify incoming JSON payloads.

3. Constraints & Dependencies:

- **Rate Limiting & Anti-Scraping:** Instagram employs aggressive anti-scraping measures that can block IP addresses or request accounts to solve Captchas. To ensure 100% availability and prevent blocks, InstaGuard operates strictly on user-supplied parameters rather than automated backend scraping.
- **Data Completeness:** The accuracy of predictions depends on the presence of public metadata. Profiles that hide their metrics or are completely locked down can limit the input space.
- **Technical Stack:** Python 3.x, Pandas & NumPy (data loading & arrays), Scikit-Learn (preprocessing & splitting), XGBoost (ensemble classification trees), and Flask (backend serving layout).

Phase 2: System Design & Methodology

The system architecture separates the heavy computational training process from the lightweight serving layer. This division ensures that the production web server remains fast and responsive. The lifecycle flow is detailed below:

Phase / File	Stage	Description & Data Flow
Offline: dataset.csv	Data Source	Contains raw profile stats of 576 accounts.
Offline: data_loader.py	Ingestion	Loads raw CSV into Pandas DataFrame.
Offline: feature_engineer.py	Feature Prep	Calculates follower ratios & bio boolean indicators.
Offline: preprocessor.py	Splitting	Separates target class and splits 80% train / 20% test.
Offline: model_trainer.py	Training	Trains XGBoost Classifier and saves model.pkl to disk.
Offline: visualiser.py	Evaluation	Runs predictions on test set and outputs accuracy.
Online: app.py / Web UI	Web Interface	User inputs stats on dashboard; receives instant predictions.
Online: predictor.py	Inference	Loads model.pkl, executes feature ratios, predicts output class.

Feature Engineering Theory:

To expose anomalies typical of bot behavior, 4 specialized features are engineered:

- **Follower-to-Following Ratio:** Real users typically establish mutual connections or follow a selective set of accounts, resulting in balanced ratios. In contrast, bot accounts mass-follow thousands of users to gain follow-backs while having very few followers of their own. Formula: *followers / (follows + 1)*. (The +1 smoothing term avoids division-by-zero).
- **Posts-to-Following Ratio:** Flags inactive accounts that follow massive numbers of users. Formula: *posts / (follows + 1)*. • **Has Bio:** Boolean flag (1/0) indicating whether description length > 0. Automated bot creators often skip writing profile biographies to save creation time.
- **Has Full Name:** Boolean flag (1/0) showing if full name words > 0.

Machine Learning Algorithm: XGBoost

We chose **XGBoost (Extreme Gradient Boosting)** as our classification model. XGBoost builds an ensemble of weak decision trees sequentially, with each subsequent tree focusing on correcting the errors made by its predecessors. It is highly optimized for tabular datasets, handles missing values naturally, prevents overfitting via regularization parameters, and requires no feature scaling (unlike SVMs or neural networks).

Hyperparameter Settings: *n_estimators = 100* (number of trees), *max_depth = 6* (limits depth to prevent overfitting), *learning_rate = 0.3* (step size shrinkage), and *eval_metric = 'logloss'*.

Phase 3: Development & Implementation (Engineering Logbook)

1. Codebase Layout & Script Walkthrough:

```
d:\fake_profile_detector\
 app.py                     # Flask web server and backend logic
 config.py                  # Hyperparameter configuration
 main.py                    # Master pipeline runner
 src/
     data_loader.py         # CSV file reader
     feature_engineer.py    # Automated ratio computations
     preprocessor.py        # Train/test set splitting
     model_trainer.py       # XGBoost classifier training
     visualiser.py          # Model evaluation reporting
     predictor.py           # On-the-fly requests parser
```

2. Log of Development Hurdles & Bug Resolution:

Hurdle A (Division-by-Zero Exceptions): Accounts with 0 following counts caused program crashes and produced NaN values during feature extraction. We resolved this by adding a +1 smoothing factor to the denominators: df['follower_following_ratio'] = df['#followers'] / (df['#follows'] + 1). This guarantees continuous, valid numerical ranges across all users.

***Hurdle B (Frontend-to-Model Feature Mapping*):** Flask clients submit string fields (like biography, username, fullname), but the XGBoost model expects numeric metrics. To bridge this gap without scraping backend endpoints, we built string-to-numeric converters in app.py that dynamically compute digit-ratios, word counts, and bio lengths on incoming requests.

Hurdle C (Robust Input Type Parsing): Users leaving input fields blank or entering invalid non-numeric strings originally triggered Flask internal server errors. We resolved this by writing a safe_int() wrapper inside app.py that catches TypeErrors and ValueErrors, defaulting empty strings or invalid inputs to 0 safely.

Hurdle D (Model Loading Dependency Bug): Running the web application prior to executing the ML training pipeline originally caused server crashes due to a missing model.pkl file. We fixed this by placing a try-except block at server startup that checks for FileNotFoundError, logs a user warning, and disables inference cleanly rather than hard-crashing.

3. Project Configuration Settings:

- **XGBoost Model Tuning:** Hyperparameters were locked in *config.py*: *n_estimators=100*, *max_depth=6*, and *eval_metric='logloss'*. The explicit evaluation metric suppresses warnings in newer XGBoost versions and specifies binary cross-entropy loss optimization.

- **Path Portability:** The config file dynamically resolves paths using *os.path.dirname(os.path.abspath(__file__))*, ensuring the training and serving scripts function seamlessly on any user machine without hardcoding path structures.
- **Dependency Separation:** An isolated Python virtual environment (*venv/*) was established, separating core libraries (Pandas, Scikit-Learn, XGBoost, and Flask) from global system packages.

4. Technical Breakthroughs:

- **XGBoost Classifier Integration:** Achieved a robust **90.52% accuracy**, significantly outperforming linear models by capturing non-linear interactions (e.g., follower/following ratios and digit counts).
- **Zero Training-Serving Skew:** Implemented a unified feature engineering module (*src/feature_engineer.py*) shared between the training script and the prediction engine, guaranteeing identical feature scaling and data columns in both training and inference contexts.
-

Phase 4: Testing & Data Collection

1. Data Collection & Dataset Composition:

The dataset used to train and validate InstaGuard consists of a benchmark corpus of **576 unique Instagram accounts**. To ensure robust classifier convergence and prevent model bias, the dataset is perfectly balanced: it contains exactly **288 legitimate profiles (Class 0)** and **288 fake/spam profiles (Class 1)**.
The raw attributes collected cover three key profile dimensions:
- *Binary Indicators:* presence of a profile picture, external bio URL, and account privacy setting.
- *Integer Metadata Counts:* total postings count, follower count, and follows count.
- *Text-Derived Attributes:* bio biography character count, full name word count, and username/fullname digit-to-length ratios.

During ingestion via *data_loader.py*, raw CSV data tables are parsed into Pandas dataframes. The data pipeline cleans the records and verifies feature data types. It ensures all input vectors contain only numerical values (integers or floating-point decimals) prior to training, preventing data formatting exceptions.

2. Testing & Validation Methodology:

To perform strict validation, the dataset was split using an **80/20 partitioning strategy**. A total of **80% (460 rows)** of the data was allocated to the training partition, allowing the XGBoost decision trees to learn behavioral feature boundaries. The remaining **20% (116 rows)** of the records was withheld as a hidden validation partition, simulating real-world unseen production traffic.
To guarantee exact reproducibility across runs, the partition split was locked using a random seed value of *random_state = 42*. The validation metrics are evaluated by computing:
- **Precision:** measures prediction exactness (out of all profiles flagged as fake, what percentage are actually fake). Formula:

TP / (TP + FP).

- **Recall:** measures prediction completeness (out of all actual fake profiles, what percentage did the model catch). Formula:

TP / (TP + FN).

- **F1-Score:** the harmonic mean of precision and recall. Formula: *2 * (Precision * Recall) / (Precision + Recall)*.
- **Support:** the real counts of occurrences in the evaluation split (63 real, 53 fake).

3. Quantitative Model Metrics:

The model was evaluated on the hidden validation set, obtaining an **overall accuracy of 90.52%** (105 out of 116 cases classified correctly):

Metric	Real Profiles (Class 0)	Fake Profiles (Class 1)	Weighted Avg
Precision	0.89	0.92	0.91
Recall	0.94	0.87	0.91
F1-Score	0.91	0.89	0.90
Support	63	53	116

Analysis of Metrics:

- **High Recall for Real Profiles (0.94):** This indicates that only 6% of real users are accidentally flagged as bots (low false positive rate). This is highly desirable in production systems, as blocking legitimate users causes frustration.
- **High Precision for Fake Profiles (0.92):** This means that when the model flags a profile as 'Fake', it is correct 92% of the time.

User Acceptance Testing (UAT) Verification Cases

To verify model robustness and boundary conditions, UAT validation cases were executed. The table below represents the **complete list of all 11 features** mapped for each scenario to demonstrate how raw metrics affect the model's ultimate prediction:

Parameter / Output	Case 1: Standard User	Case 2: Follow Spammer	Case 3: Empty Bio Bot	Case 4: Balanced Private
Profile Picture?	Yes (1)	No (0)	No (0)	Yes (1)
Username Digit Ratio	0.00	0.35	0.50	0.00
Full Name Words	2	0	1	2
Full Name Digit Ratio	0.00	0.00	0.00	0.00
Name == Username?	No (0)	No (0)	No (0)	No (0)

Bio Char Length	45	0	0	60
External URL in Bio?	Yes (1)	No (0)	No (0)	No (0)
Is Account Private?	No (0)	No (0)	No (0)	Yes (1)
Number of Posts	120	0	1	45
Number of Followers	450	1	10	150
Number of Follows	380	4500	250	180
Expected Class	Real	Fake	Fake	Real
ML Prediction	Real (Class 0)	Fake (Class 1)	Fake (Class 1)	Real (Class 0)
UAT Verdict Status	**Passed**	**Passed**	**Passed**	**Passed**

Future Recommendations & Extensions:

- **Natural Language Processing (NLP):** Incorporate sentiment and keyword analysis on the biography string to detect spam links or fraudulent offers.
- **Visual CNN Layer:** Add a secondary convolutional neural network to evaluate the profile picture for generic stock photos or missing image features.
- **Dynamic API Ingestion:** Integrate a background queue to query Instagram's public JSON endpoint to automatically load metrics by username.

Conclusion:

InstaGuard successfully demonstrates that machine learning, specifically the XGBoost Classifier, is an effective tool for identifying automated bot accounts on Instagram using only publicly available metadata. By engineering features such as the follower/following and posts/following ratios, the model successfully differentiates complex bot patterns from legitimate user accounts with an out-of-sample accuracy of 90.52%. The serialized model loading architecture allows for rapid, sub-100ms predictions in production, making it a highly scalable and practical defense mechanism against social media spam and fraud